

Agency Client and Campaign Tracker - Project Report

Project Report — Programming in Java (Flipped Course Evaluation)

Submitted by: [Your Name] | Registration No: [Your Reg. No] | Course: Programming in Java

- [1. Introduction](#)
- [2. Problem Statement](#)
- [3. Objectives](#)
- [4. Functional Requirements](#)
- [5. Non-Functional Requirements](#)
- [6. Design](#)
 - [6.1 Use Case Diagram](#)
 - [6.2 Class Diagram](#)
 - [6.3 Process Flow — Generate Invoice](#)
- [7. Implementation Details](#)
- [8. Testing](#)
- [9. Challenges Faced](#)
- [10. Learnings](#)
- [11. Future Enhancements](#)
- [12. References](#)

1. Introduction

Small marketing agencies often track clients, campaigns, and invoices manually, which makes it easy to lose track of what's active and what's still unpaid. This project is a simple command-line Java application that lets agency staff record clients, the campaigns run for them, and the invoices raised against them, with all data saved locally between runs.

2. Problem Statement

Manually tracking clients, campaigns, and billing across notes or spreadsheets leads to lost context: no easy way to see which campaigns are active, or how much money has been collected versus is still owed. This project addresses that with a lightweight, file-based console tool that needs no database or internet connection.

3. Objectives

1. Let staff record and view clients.
2. Let staff create campaigns tied to a client and view them.
3. Let staff generate invoices and mark them paid.
4. Show simple reports on totals, revenue, and outstanding dues.
5. Persist all data between runs using a local file.

4. Functional Requirements

#	Module	Description
FR1	Client Management	Add a client (name, industry, email); view all clients.
FR2	Campaign Management	Create a campaign for an existing client (title, budget); view all campaigns.
FR3	Invoicing	Generate an invoice for a client; mark an invoice as paid; view all invoices.
FR4	Reports	Show total clients, total campaigns, revenue collected, and outstanding dues.

Input is taken through a `Scanner`-based numbered menu; output is printed as plain text lines.

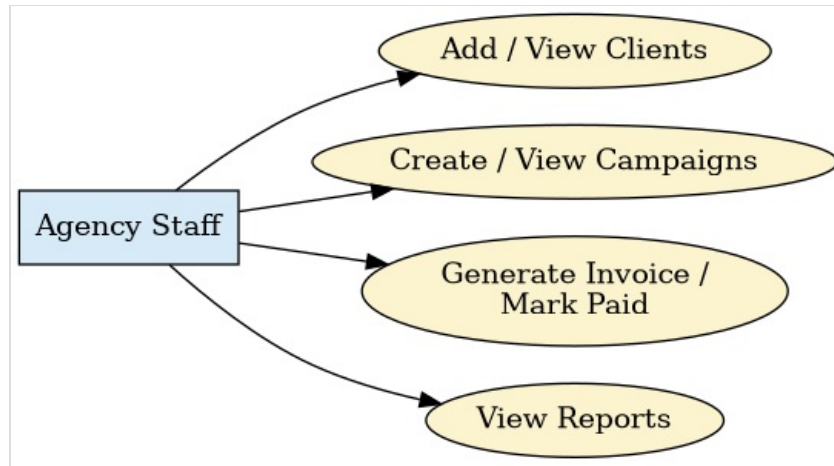
5. Non-Functional Requirements

Requirement	How it is addressed
Reliability	A custom <code>NotFoundException</code> is thrown and caught whenever a campaign or invoice references a client ID that doesn't exist, instead of the program crashing. Invalid number input (e.g. a non-numeric budget) is also caught.
Usability	Every screen uses the same simple numbered-menu pattern.
Maintainability	Each module (clients, campaigns, invoices, reports) has its own menu method, keeping related logic together.
Persistence	All data is saved to <code>agency_data.txt</code> on exit and reloaded on the next run, so nothing is lost between sessions.

6. Design

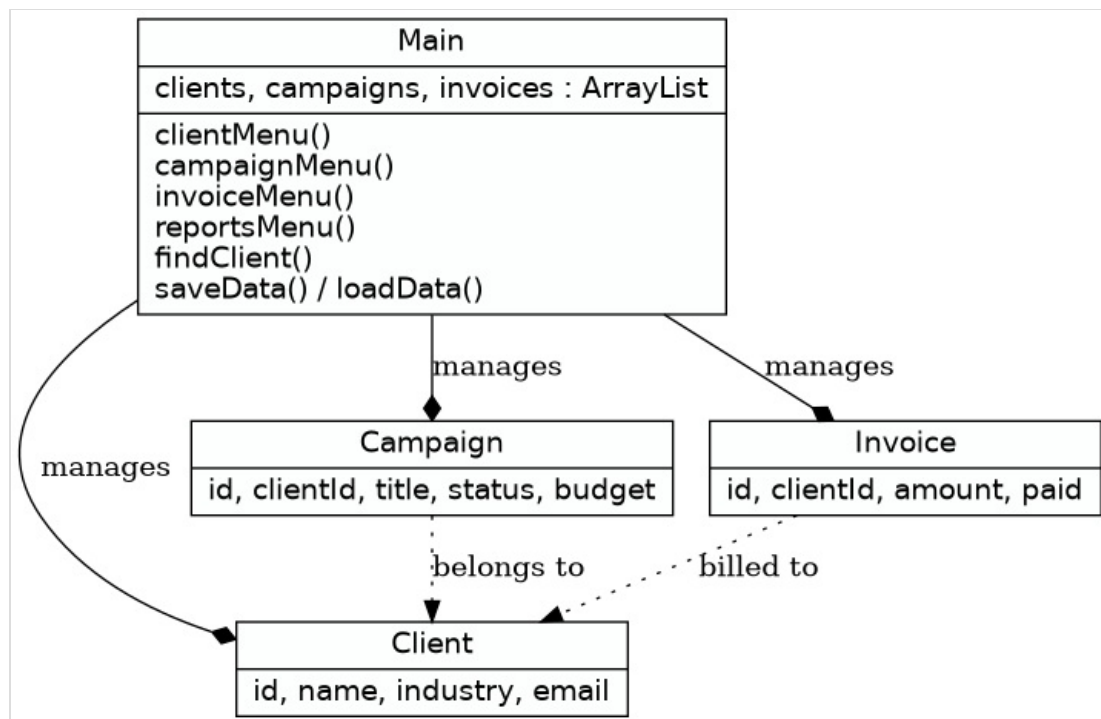
The application is intentionally kept to a single class, `Main`, with three small data classes (`Client`, `Campaign`, `Invoice`) nested inside it. Everything is stored in `ArrayList` s in memory while the program runs, and written out to one text file when the user exits.

6.1 Use Case Diagram



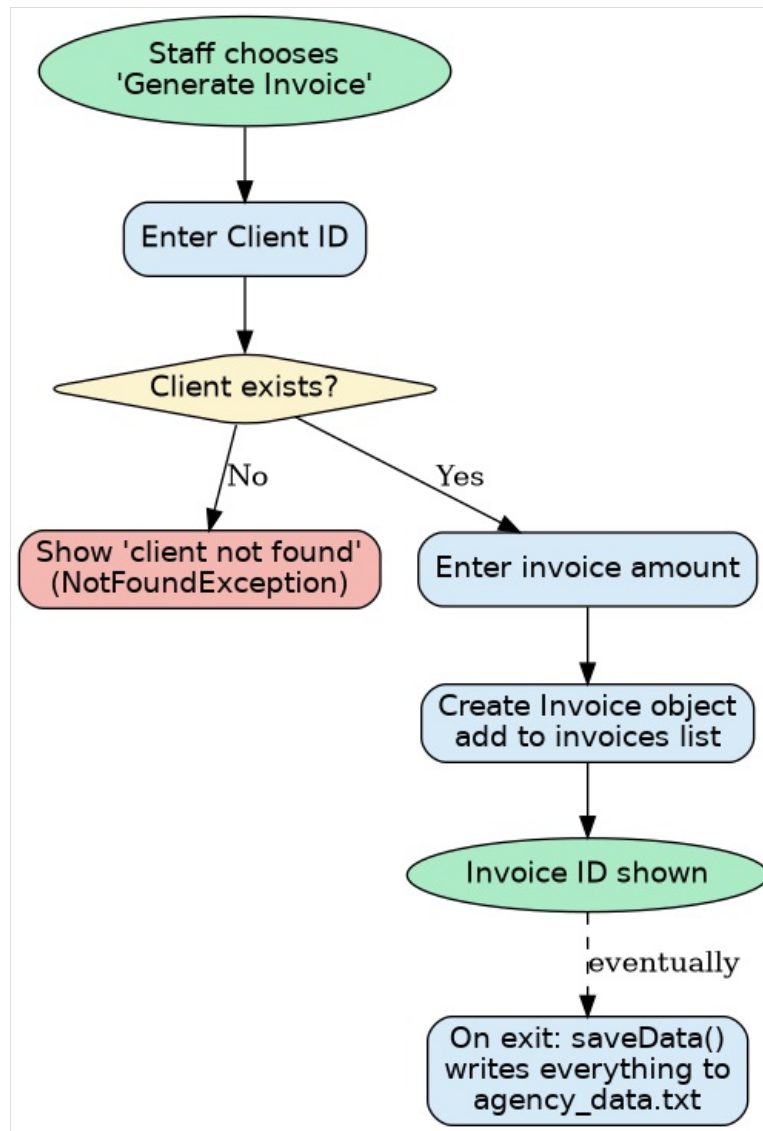
Use Case Diagram

6.2 Class Diagram



Class Diagram

6.3 Process Flow — Generate Invoice



Generate Invoice Flow

7. Implementation Details

- **Language:** Java 17+, standard library only.
- **Data classes:** `Client` (id, name, industry, email), `Campaign` (id, clientId, title, status, budget), `Invoice` (id, clientId, amount, paid).
- **Storage:** three `ArrayList`s (`clients`, `campaigns`, `invoices`) held as static fields in `Main`.
- **File format:** each line in `agency_data.txt` starts with `CLIENT`, `CAMPAIGN`, or `INVOICE` followed by comma-separated fields, so `loadData()` can tell which kind of record each line is and rebuild the right object.
- **ID generation:** simple incrementing counters (`clientCounter`, `campaignCounter`, `invoiceCounter`) produce IDs like `C1`, `CM1`, `INV1`.
- **Error handling:** `findClient()` throws `NotFoundException` when a client ID doesn't exist, caught by the calling menu method, which prints a friendly message instead of crashing.

8. Testing

The application was tested manually by running through the full workflow: adding a client, creating a campaign against it, generating and paying an invoice, checking the reports update correctly,

and confirming an invalid client ID produces a clean error message rather than a crash. Restarting the app after saving was also checked to confirm previously entered data reloads correctly from `agency_data.txt`.

9. Challenges Faced

- Deciding on a simple, parseable file format that could tell three different record types apart on load — solved by prefixing each line with `CLIENT`, `CAMPAIGN`, or `INVOICE`.
- Making sure a campaign or invoice could never be created for a client ID that doesn't exist, without over-complicating the code — solved with one small `findClient()` helper that throws a custom exception.

10. Learnings

- Practiced using classes and objects to represent real-world entities (Client, Campaign, Invoice).
- Practiced custom exception handling with a purpose-built `NotFoundException`.
- Practiced reading and writing structured data to a text file using `BufferedReader` and `BufferedWriter`.

11. Future Enhancements

- Add a login step for staff.
- Support editing or deleting existing clients and campaigns.
- Add due dates and automatic overdue detection for invoices.

12. References

1. Oracle, *Java Platform, Standard Edition Documentation* — <https://docs.oracle.com/en/java/javase/>
2. Oracle, *java.io Package Documentation* — <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/io/package-summary.html>